

Towards Propositional Reasoning: Evaluating Design Choices in an LLM-Based NL-to-Logic Translation Pipeline

Anonymous authors

Paper under double-blind review

Abstract

One important approach to improving the reasoning abilities of Large Language Models (LLMs) is to convert natural-language questions into logical propositions so that reasoning is performed via external tools such as logic solvers. However, translating ambiguous natural language into logical form is a known bottleneck that can introduce errors. Here, we study an LLM-based pipeline that, given a text, first converts the text into a propositional knowledge base consisting of primitive propositions, their natural-language interpretations, and propositional formulas capturing the logical constraints expressed in the text. We then answer natural-language queries by translating them into a propositional formula over that knowledge base and solving it with a SAT solver. We study how different design choices affect end-to-end reasoning and translation accuracy. In particular, we study the impact of: (i) using OpenIE triples to support text-to-logic conversion; (ii) deterministic knowledge-base enrichment; (iii) shortcut-based query translation that can bypass some LLM calls; and (iv) query expansion with multiple variants and majority voting. Using a fractional factorial experimental design, we identify the effects of the different components and combinations. In particular, where additional complexity leads to diminishing returns.

1 Introduction

Reliable reasoning over text remains a core challenge in AI. Given a text, users need to determine whether a statement is entailed, contradicted, satisfiable, or completely irrelevant. Large language models (LLMs) provide natural-language interfaces for such tasks, but they conflate two fundamentally different problems: *language semantics* (extracting meaning from text) and *logical reasoning* (deriving valid conclusions from premises stated in the text). This conflation is the source of their unreliability. LLMs hallucinate facts, preferring external knowledge to the input document (Ji et al., 2023); miss information in long contexts, prioritizing the start and end while neglecting the middle (Liu et al., 2024a); and produce plausible but logically unsound conclusions (Huang & Chang, 2023).

A recent survey identifies neuro-symbolic methods, which separate language semantics from logical reasoning, as a promising paradigm for improving reasoning in LLMs (Yang et al., 2025). One class of neuro-symbolic methods translates natural language into formal representations, delegates inference to symbolic solvers, and interprets results—the LLM serves as a bridge between natural language and formal logic. However, these approaches are typically evaluated on benchmarks that favor the specific formalism they adopt, making cross-method comparison difficult. FOLIO (Han et al., 2024) and ProofWriter (Tafjord et al., 2021) are constructed around first-order logic; LogiQA2.0 (Liu et al., 2023) and LogicBench (Parmar et al., 2024) frame propositional and multi-step logical reasoning as multiple-choice questions; and DocNLI (Yin et al., 2021) and ContractNLI (Koreeda & Manning, 2021) measure textual entailment (instead of logical) over longer text.

To better understand the transition from natural language to logic, we study a configurable pipeline that converts text into a propositional knowledge base of atomic propositions along with propositional formulas representing hard constraints, and then answers queries via

a SAT solver. Using four binary design factors, a full factorial experiment would require $2^4 = 16$ runs per query—costly, especially when each run involves potentially multiple LLM calls. We adopt a 2_{IV}^{4-1} fractional factorial design (Box et al., 2005), which reduces the experimental runs to 8 while preserving the ability to estimate all main effects and most two-factor interactions. This allows us to identify the most influential pipeline components with statistical rigor.

We evaluate this pipeline on three datasets: LogicBench, a variation of DocNLI, and a custom hand-written “Alice” one. LogicBench tests reasoning over short premises, while DocNLI evaluates entailment over longer documents. Because DocNLI measures textual rather than logical entailment, we combine an LLM and human expertise to generate queries that require logical inference from the source text. These two datasets purposefully avoid modal language, focusing on classical propositional reasoning. To evaluate reasoning in the presence of modalities, we introduce the third set of questions, called Alice, which includes modal words such as “always,” “usually,” “rarely,” and “never.” Different configurations may favor different datasets, and our factorial analysis identifies how the components behave across this diverse evaluation suite.

2 Related works

2.1 Neuro-symbolic solvers

Recent work combines LLMs with symbolic solvers to improve logical reasoning. Logic-LM (Pan et al., 2023) introduced a framework that uses LLMs to translate natural language text and queries into a first-order logic (FOL) knowledge base, iteratively solve FOL syntax issues refining from error messages, and offloads to an SMT solver to determine entailment. Logic-LM++ (Kirtania et al., 2024) extends this with multi-step refinement using pairwise comparisons to evaluate candidate revisions, achieving 78.9% on FOLIO and 79.7% on ProofWriter. LINC (Olausson et al., 2023) takes a similar approach but uses the Prover9 theorem prover for deductive inference and, instead of fixing incorrect FOL syntax, LINC samples (at default 10) candidate translations and uses a majority voting strategy. LINC reaches 87% on ProofWriter with GPT-4, outperforming chain-of-thought prompting by 10–38% absolute depending on model size.

Both systems formalize problems per query on short benchmarks; in contrast, our approach studies the effect of first converting the initial document into a set of logical propositions and constraints, a so-called *logification*, of the document. Then, it supports repeated querying over longer texts. A major source of error, as noted in the cited works, arises from the FOL formulation. FOL translation requires choosing predicate arities, variable scopes, and quantifier structures—decisions that introduce semantic ambiguity beyond the LLM’s competence. Logic-LM’s execution rate drops from 99% (propositional-like ProofWriter) to 79.9% (FOL-based FOLIO). And LINC’s error analysis attributes failures primarily to the FOL translation process—implicit information loss and representation choices—rather than the symbolic reasoning itself. We therefore deliberately restrict ourselves to propositional logic.

Our approach also fits within a broader line of work that augments LLMs with external tools to offload computational reasoning. For example, PAL (Gao et al., 2023) and Program-of-Thought (Chen et al., 2023) generate Python code for arithmetic reasoning, delegating calculation to an interpreter while the LLM handles problem decomposition. SatLM (Ye et al., 2023) uses declarative prompting to cast problems as satisfiability queries, achieving state-of-the-art on LSAT and BoardgameQA by offloading inference to an automated theorem prover. Logic-of-Thought (Liu et al., 2024b) injects propositional logic into prompts as an augmentation layer: it extracts logical expressions, extends them via inference rules, translates them back to natural language, and appends this enriched context to the original prompt for the LLM to reason over. All of these approaches outperform CoT baselines by separating what LLMs do well (processing language) from what they do poorly (valid, intricate reasoning).

2.2 Semantic similarity and textual entailment

Our pipeline leverages pretrained models for semantic retrieval and entailment scoring. Sentence-BERT (SBERT) (Reimers & Gurevych, 2019) modifies the BERT architecture with siamese networks to produce fixed-size sentence embeddings that capture semantic similarity; cosine similarity between embeddings enables efficient retrieval without cross-encoding every pair. For textual entailment, we use a DeBERTa-v3-large cross-encoder fine-tuned on NLI datasets (He et al., 2021). Unlike SBERT’s bi-encoder, a cross-encoder jointly encodes a premise-hypothesis pair and outputs probabilities over entailment, contradiction, and neutral. While more accurate, it is computationally expensive so we use SBERT for retrieval and cross-encoder NLI for scoring.

For syntactic analysis and lexical knowledge, we rely on established NLP tools. SpaCy (Honnibal et al., 2020) provides tokenization, part-of-speech tagging, and dependency parsing, which we use for negation detection and grammatical analysis. WordNet (Miller, 1995) supplies synonym sets and antonym relations, enabling our modal-opposite detection and query expansion mechanisms. Stanford CoreNLP’s OpenIE (Angeli et al., 2015), accessed via Stanza (Qi et al., 2020), extracts subject-predicate-object triples from text.

3 System architecture

Our pipeline follows a modular design with three components: (1) `from_text_to_logic`, which *logifies* text into atomic propositions, their translation into natural language, and hard constraints inferred from the text; (2) `interface_with_user`, which handles translation from natural language queries to propositional formulas over the logified atomic propositions; and (3) `logic_solver`, which performs reasoning via a SAT solver. This modularity facilitates our fractional analysis with four boolean factors, two factors for enriching the logification and the other two for enriching queries.

3.1 From text to logic

We implement the extraction of natural language text into a logified structure in up to four stages. As part of our factorial experiment, this extraction depends on two binary factors described in Table 1: `use_openie`, which enriches the original text with subject-predicate-object triples, and `use_enrichment_kb` which enriches the original logified file to better support both modal questions and related propositional queries.

Text-to-knowledge-base construction stage		
	Factor	Description
T1	<code>use_openie</code>	Supplies Stanford CoreNLP OpenIE relation triples as auxiliary input to the LLM during text-to-logic conversion.
T2	<code>use_enrichment_kb</code>	Applies enrichment (modal-pair verification, negation detection, finite-domain auxiliaries, and conflict resolution) to expand and validate the knowledge base.

Table 1: Binary factors for the text-to-logic stage of our factorial experiment. Each factor enables or disables a distinct pipeline component.

3.1.1 Relation triple extraction

We use Stanford CoreNLP’s OpenIE system to extract subject-predicate-object triples from the text. We first apply native Stanza coreference resolution to ensure that triples consistently refer to canonical entities rather than pronouns, e.g., replacing “they” with “Alice”. There are cases where the grammatical structure fails for OpenIE, e.g. “Alice sleeps soundly” yields no OpenIE triple, so we extract the subject-verb pair from Stanza’s dependency parse.

3.1.2 LLM logic conversion

We input the original text T and, when `use_openie` is enabled, its extracted triples to GPT-5.2 (with medium reasoning effort) using a few-shot Chain-of-Thought (CoT) prompt. The LLM identifies atomic propositions $\mathcal{P}(T) := \{P_1, P_2, \dots, P_n\}$ and a collection of propositional formulas over $\mathcal{P}(T)$ representing constraints $\mathcal{C}(T) := \{C_1, C_2, \dots, C_m\}$. For each constraint, the LLM assigns an initial weight $w \in [0, 1]$ reflecting confidence that the constraint is obligatory; for example, if $w_1 = 1$ then the LLM determined that the constraint C_1 is enforced from the text. Note, we use two different prompts according to whether we are testing the Alice dataset or not. Only in the Alice dataset do the atomic propositions contain modal expressions representing degrees of uncertainty (e.g., “must,” “likely,” “may”). For the other datasets, the LLM strips all modality away from the atomic propositions, and the assigned weight reflects this degree.

The LLM generates an initial JSON structure containing: (i) primitive propositions with their natural-language translations and textual evidence, and (ii) constraints with their logical formulas, translations, evidence, and initial weights (see Appendix A.1 for the full prompt).

3.1.3 Enriching the knowledge base

When `use_enrichment_kb` is enabled, we apply four deterministic enrichment steps to the initial logified structure:

1. **Modal-pair verification:** For each atomic proposition containing a modal word (e.g., “always,” “usually,” “rarely”), we add another atomic proposition, if missing, that removes the modality. For example, if “*Alice rarely studies late at night*” is an atomic proposition in the original logified JSON, we insert a new one “*Alice studies late at night.*” In particular, this step only applies to the Alice dataset because for the other datasets, the modalities are already stripped.
2. **Modal-opposite detection:** Using WordNet antonyms, we identify propositions with opposing modal words (e.g., “typically” vs. “rarely”) over the same base event. When such pairs are found, we add mutual exclusion constraints. For instance, if the text contained both P_i : “*Alice typically studies hard*” and P_j : “*Alice rarely studies hard,*” we would add $P_i \Rightarrow \neg P_j$ and $P_j \Rightarrow \neg P_i$ to enforce that these cannot both hold.
3. **Negation detection:** We use SpaCy to detect whether the original logified knowledge base includes atomic propositions that are negations (e.g., “not,” “never,” “doesn’t”). If present then we extract the affirmative clause, and add it as a new atomic proposition if its not already there determined with SBERT.
4. **Finite-domain auxiliaries:** Propositional logic lacks the expressivity to represent finite-domain constraints directly (e.g., “*Alice studies exactly one major*”). To partially address this, we detect propositions involving values from known finite domains (e.g., days of the week) and generate auxiliary propositions for alternative values, adding mutual exclusion constraints between them.

Finally, we apply NLI-based conflict resolution to detect and resolve cases of inconsistency in the resulting logified knowledge base. If a proposition and its negation are asserted as constraints, we keep the proposition with higher NLI entailment score from the source text.

3.1.4 Final logified structure

We determine which weighted constraints to keep as hard constraints for the final logified JSON. For each constraint C_i with LLM-assigned weight w_i^{llm} , we compute an NLI entailment score w_i^{nli} as follows. We retrieve the top-10 most similar propositions using SBERT and measure the maximum entailment probability using a cross-encoder NLI model. The combined score is the product $w_i = w_i^{llm} w_i^{nli}$. Constraints which exceed a fixed hardness threshold ($w_i \geq 0.9$ in our experiments) become the final hard constraints; the remaining become soft constraints. Entries in the final JSON have a template shown in Appendix A.2.

3.2 Interface with user

Once we have logified the text, we can process queries about that text and solve them with a logic solver. The purpose of this module is to translate a given query into a propositional formula. Translation depends on two binary factors described in Table 2: `use_shortcuts`, which avoids some cases of calling an LLM, and `expand_query`, which generates synonymous queries and uses a voting strategy for translation.

Query-to-formula translation stage		
	Factor	Description
Q1	<code>use_shortcuts</code>	Activates deterministic shortcut detectors (modal opposites, lexical antonyms, and implication contradictions) that bypass the LLM when certain patterns are found in the query.
Q2	<code>expand_query</code>	Creates query alternatives using WordNet synonym variants filtered by SBERT similarity, and enables majority-vote ensembling over multiple LLM calls to select the final logical formula. Both mechanisms are activated jointly.

Table 2: Binary factors for the query-to-formula translation stage of our factorial experiment.

3.2.1 Query translation

Given a natural-language query and the logified knowledge base, we first determine the query type. If the query is phrased as a question, we convert it to a declarative statement—for example, “Does Alice study computer science?” becomes “Alice studies computer science.” We then retrieve the top-10 closest propositions to this statement using cosine similarity over SBERT embeddings. This retrieved subset is provided to the LLM (see Appendix A.4 for the prompt), which outputs either a propositional formula over the retrieved propositions, or NONE if the query is unrelated to the source text.

3.2.2 Shortcut detectors

When `use_shortcuts` is enabled, deterministic shortcut detectors can resolve certain queries before invoking the LLM. These include detecting contradictions between modal expressions such as “must” and “must not,” identifying direct lexical antonyms using WordNet (for example, “always” versus “never”), and recognizing implication-based contradictions such as “X requires Y” versus “X does not require Y.” When a shortcut fires, it bypasses LLM translation entirely, reducing latency and avoiding potential translation errors.

3.2.3 Query expansion and voting

When `expand_query` is enabled, we apply two mechanisms to improve translation robustness. First, synonym expansion generates multiple queries using WordNet synonyms filtered by SBERT similarity to ensure semantic fidelity; each variant is translated independently by the LLM. Second, majority voting selects the final formula by vote over the multiple candidate translations, weighted by translation confidence. This ensemble approach reduces sensitivity to specific query phrasings.

3.3 Logic solver

We use PySAT’s RC2 solver (Ignatiev et al., 2019) to determine results. To check entailment, we add $\neg Q$ as a hard constraint and test satisfiability; if the resulting formula is unsatisfiable, then the knowledge base entails Q . To check contradiction, we instead add Q as a hard constraint; if this makes the formula unsatisfiable, then the knowledge base entails $\neg Q$. To check uncertainty, we verify that neither case holds—that is, both Q and $\neg Q$ are

consistent with the knowledge base. For queries where the translation returns NONE, we report NOT_MENTIONED: the query concerns content outside the knowledge base’s scope.

4 Experiments

4.1 Datasets

4.1.1 LogicBench

(Parmar et al., 2024) is a comprehensive logical reasoning benchmark comprising 25 distinct inference patterns across propositional, first-order, and non-monotonic logics. We use the LogicBench(Eval) subset, in particular binary question-answer instances. Each example presents a natural language context encoding logical premises and asks whether a conclusion follows. The dataset tests fundamental inference rules including modus ponens, modus tollens, disjunctive syllogism, hypothetical syllogism, and constructive/destructive dilemmas. We evaluate on 320 examples balanced across 8 propositional logic patterns.

4.1.2 DocNLI

(Yin et al., 2021) is a document-level natural language inference dataset containing premise-hypothesis pairs from diverse sources, including news articles, Wikipedia passages, and other document corpora, labeled for entailment vs. not entailment. Because these queries target textual rather than strict logical entailment, we regenerate them to better exercise our neuro-symbolic pipeline. We prompt an LLM (see Appendix A.3) to generate 16 single-sentence query per text: 4 *entailment*, 4 *contradiction*, 4 *uncertain*, and 4 *not mentioned*. This yields 320 queries across 10 texts, evenly distributed among the four output categories. The new queries are then evaluated by the authors to detect any imprecision.

4.1.3 Alice

“Alice” is a controlled dataset we constructed to evaluate reasoning capabilities in everyday scenarios. These data consist of a short document describing “Alice,” a university student, with 20 hypothesis statements spanning four inference types: direct entailment, contradiction, neither entailed nor contradicted, and statements not mentioned in the source. Unlike the other works, this dataset supports reasoning over modal language such as “always,” “usually,” “rarely,” and “never,” allowing us to test the effect of uncertainty on translation.

4.2 Experimental approach: fractional design

Run	Q_1	Q_2	T_1	T_2
1	—	—	—	—
2	+	—	—	+
3	—	+	—	+
4	+	+	—	—
5	—	—	+	+
6	+	—	+	—
7	—	+	+	—
8	+	+	+	+

Table 3: 2^{4-1}_{IV} fractional factorial design with generator $T_2 = Q_1 Q_2 T_1$.

We recall that fractional factorial designs are a class of experimental designs that enable efficient exploration of multiple factors using only a subset of all possible combinations.

We evaluate our pipeline using four binary factors: Two factors affecting the text-to-logic conversion (T_1 : use_openie, T_2 : use_enrichment_kb; see Table 1) and the other two factors involved on the query translation (Q_1 : use_shortcuts, Q_2 : expand_query; see Table 2). We

249 adopt a 2_{IV}^{4-1} fractional factorial design with generator $T_2 = Q_1 Q_2 T_1$, yielding 8 runs per
 250 query summarized in Table 3.

251 4.3 Results

252 For our modification of DocNLI, Figure 1 shows that overall accuracy varies only modestly
 253 across runs, remaining roughly in the 0.43–0.51 range. This suggests that no single configura-
 254 tion clearly dominates end-to-end performance on DocNLI. The class-wise breakdown,
 255 however, reveals larger differences: *uncertain* and *not mentioned* are consistently easier than
 256 *entailment* and *contradiction*. This pattern indicates that the pipeline is better calibrated for
 257 recognizing non-entailing cases than for recovering logically supported or contradicted
 258 statements.

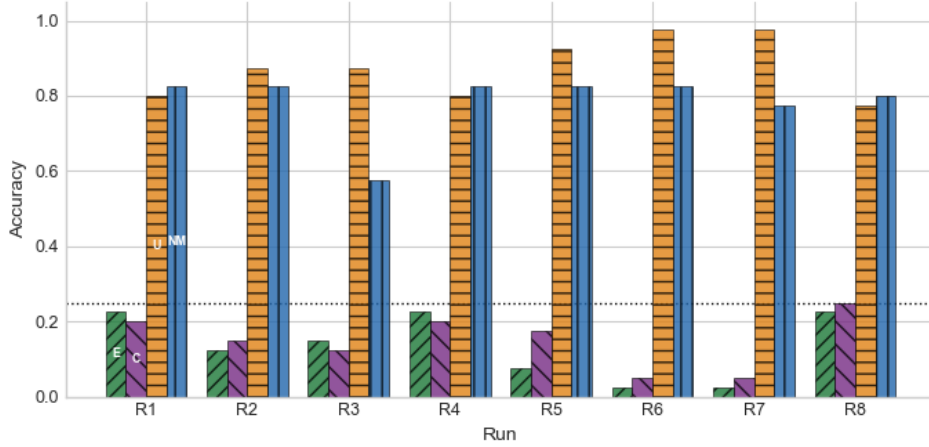


Figure 1: Per-category classification accuracy on the DocNLI edited dataset across the eight runs of the 2_{IV}^{4-1} fractional factorial design. Within each run group, bars represent from left to right: Entailment (green), Contradiction (purple), Uncertain (orange), and Not Mentioned (blue); category abbreviations are labelled on run R1. The dotted line marks the 25 % random-chance baseline.

259 Figure 2 summarizes the estimated main effects of the four binary factors by category. The
 260 strongest pattern is that enabling OpenIE (T_1) improves performance on the *uncertain* class,
 261 but reduces accuracy on *entailment* and *contradiction*. Query expansion and voting (Q_2)
 262 helps *entailment* and *contradiction*, but harms *not mentioned*, suggesting that paraphrase
 263 expansion can increase false alignment with the source document. By contrast, shortcut-
 264 based translation (Q_1) and knowledge-base enrichment (T_2) have smaller and more mixed
 265 effects. Overall, these results show that the design choices do not act uniformly across
 266 inference types: a component that improves one category may degrade another.

267 For LogicBench, Figure 3 in Appendix A.6 shows the main effects of the four factors
 268 separated by label. The effects are modest overall, but they differ by class: for entailment
 269 examples, *Shortcuts*, *Expand*, and *Enrich* yield small positive effects, while *OpenIE* has a
 270 slight negative effect. For not-entailment examples, *Enrich* has the largest negative effect,
 271 followed by *Expand* and *Shortcuts*, while *OpenIE* remains near zero.

272 Our analysis of 160 questions from LogicBench across all 8 configurations reveals a sur-
 273 prising and counter-intuitive pattern: SBERT confidence is negatively correlated with
 274 prediction accuracy ($r = -0.229$, $p < 0.01$). In other words, the system is systematically
 275 more confident when it makes incorrect predictions. The effect is most pronounced for
 276 missed entailments, where the system predicts *Uncertain* with the highest average SBERT
 277 confidence (0.889) despite the ground-truth label being *Yes*. Our results suggest that the
 278 translation step often produces formulas that superficially resemble the query—as reflected
 279 in high back-translation similarity—but fail to capture the semantic relationship required
 280 for entailment.

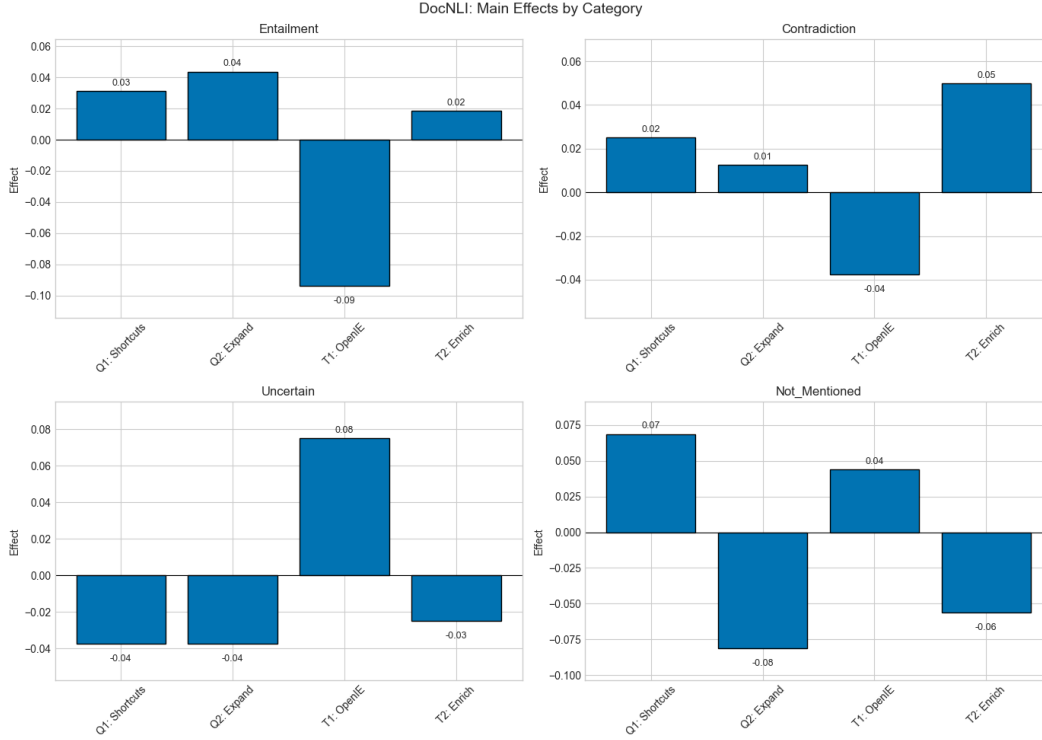


Figure 2: Estimated main effects of the four binary design factors on DocNLI, broken down by category. Positive values indicate that enabling the factor improves accuracy.

281 Similarly, we found that NLI confidence shows no significant correlation with accuracy. Pre-
 282 dictions with high contradiction confidence ($NLI < -0.6$) achieve 90.5% accuracy, whereas
 283 predictions in the neutral zone ($-0.2 < NLI < 0.2$) achieve only 24.8% accuracy. This
 284 bimodal pattern suggests that the NLI score could support selective prediction: the system
 285 can be trusted when it confidently detects contradiction, but should abstain or flag cases
 286 for review when the NLI score falls in the neutral zone. The 117 questions in this neutral
 287 region account for a disproportionate share of the errors, indicating that uncertainty in the
 288 entailment relation—rather than high confidence in the wrong direction—is the dominant
 289 failure mode.

290 For the Alice dataset, standard neural similarity metrics (SBERT, NLI) likewise fail to
 291 predict translation quality in a neuro-symbolic pipeline. In particular, SBERT confidence
 292 is negatively correlated with accuracy ($r = -0.23$, $p < 0.001$), indicating that surface-level
 293 semantic similarity is insufficient for validating logic translations. More broadly, NLI is
 294 limited here because it evaluates entailment between natural-language strings rather than
 295 between the translated logical formula and the knowledge base, and therefore has no access
 296 to whether the SAT solver will ultimately find the formula satisfiable. This highlights the
 297 need for logic-aware verification methods.

298 5 Conclusion

299 We present a configurable neuro-symbolic pipeline that converts natural language text into
 300 a propositional knowledge base and answers queries via SAT solving. Using a fractional
 301 factorial design, we conclude that the effects of individual pipeline components are not
 302 uniform: on DocNLI, different design choices improve different inference components, and
 303 no single factor consistently improves performance across all of them. An example from
 304 LogicBench in Appendix A.5, illustrates that some apparent errors may stem not from faulty

reasoning, but from differences in interpretation between our pipeline and the datasets. Namely, how results depend on how strictly benchmarks adhere to the source text.

Limitations. Our adoption of propositional logic restricts expressivity. First-order constructs such as universal and existential quantification must be flattened into finite conjunctions or disjunctions over enumerated entities. At best this inflates the number of atomic propositions, and at worse fails when domains are open-ended. By using SAT rather than SMT solvers, we lack native support for arithmetic and numerical reasoning. Statements involving comparisons, arithmetic relationships, or ordering are not easily captured.

Further work. Our framework already supports infrastructure for many-valued and modal logics beyond classical propositional logic. Extending the pipeline to leverage these capabilities would enable more faithful representation of the text we logify. We would be interested in comparing the accuracy of logifying text over those alternative logics. Additionally, we can modify our logification prompt to process much larger documents. Our current approach could scale to texts that exceed LLM context windows. The logified knowledge base, once constructed, supports efficient querying regardless of the original document length—enabling reasoning over arbitrarily large corpora.

Acknowledgments

We thank the Sundial team for their invaluable support and collaboration throughout this project.

Ethics Statement

This paper investigates AI-assisted formal reasoning over natural-language text. While neuro-symbolic pipelines can improve reliability by delegating inference to symbolic solvers, they remain vulnerable to errors in language understanding, including ambiguity, omission, and misformalization. As a result, we do not view the system described here as appropriate for fully autonomous use in high-stakes domains without human oversight.

We also disclose that AI tools were used during this project. In particular, we used the Sundial agent to assist in developing tests for the code and repository, refining implementation details, and providing feedback on parts of the manuscript. All writing was done by the authors, who take full responsibility for the final content of the paper and code.

References

- Gabor Angeli, Melvin Jose Johnson Premkumar, and Christopher D. Manning. Leveraging linguistic structure for open domain information extraction. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 344–354, 2015.
- George E. P. Box, J. Stuart Hunter, and William G. Hunter. *Statistics for Experimenters: Design, Innovation, and Discovery*. Wiley, Hoboken, NJ, 2nd edition, 2005.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 10764–10799. PMLR, 2023.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Bhattacharya, Bo Pang, Chenhao Tan, Dian Yu, and Hong Wang. FOLIO: Natural language reasoning with first-order logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 22017–22031, 2024.

- 351 Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-
352 enhanced BERT with disentangled attention. In *International Conference on Learning*
353 *Representations*, 2021.
- 354 Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy:
355 Industrial-strength natural language processing in python, 2020.
- 356 Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A
357 survey. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 1049–1065,
358 2023.
- 359 Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. RC2: An efficient MaxSAT
360 solver. *Journal on Satisfiability, Boolean Modeling and Computation*, 11(1):53–64, 2019.
- 361 Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin
362 Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language
363 generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- 364 Shashank Kirtania, Priyanshu Gupta, and Arjun Radhakrishna. LOGIC-LM++: Multi-step re-
365 finement for symbolic formulations. In *Proceedings of the 2nd Workshop on Natural Language*
366 *Reasoning and Structured Explanations (NLRSE)*, pp. 54–63. Association for Computational
367 Linguistics, 2024.
- 368 Yuta Koreeda and Christopher Manning. ContractNLI: A dataset for document-level natural
369 language inference for contracts. In *Findings of the Association for Computational Linguistics:*
370 *EMNLP 2021*, pp. 1907–1919, 2021.
- 371 Hanmeng Liu, Jian Liu, Leyang Cui, Zhiyang Teng, Nan Duan, Ming Zhou, and Yue
372 Zhang. LogiQA 2.0—an improved dataset for logical reasoning in natural language
373 understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31:
374 2947–2962, 2023.
- 375 Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni,
376 and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions*
377 *of the Association for Computational Linguistics*, 12:157–173, 2024a.
- 378 Tongxuan Liu et al. Logic-of-thought: Injecting logic into contexts for full reasoning in large
379 language models. *arXiv preprint arXiv:2409.17539*, 2024b.
- 380 George A. Miller. Wordnet: a lexical database for english. *Communications of the ACM*, 38
381 (11):39–41, 1995.
- 382 Theo X. Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua B.
383 Tenenbaum, and Roger Levy. LINC: A neurosymbolic approach for logical reasoning
384 by combining language models with first-order logic provers. In *Proceedings of the 2023*
385 *Conference on Empirical Methods in Natural Language Processing*, pp. 5153–5176, 2023.
- 386 Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empower-
387 ing large language models with symbolic solvers for faithful logical reasoning. In *Findings*
388 *of the Association for Computational Linguistics: EMNLP 2023*, pp. 3806–3824, 2023.
- 389 Mihir Parmar, Nisarg Patel, Neeraj Varshney, Mutsumi Nakamura, Man Luo, Santosh
390 Mashetty, Arindam Mitra, and Chitta Baral. LogicBench: Towards systematic evaluation
391 of logical reasoning ability of large language models. In *Proceedings of the 62nd Annual*
392 *Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13679–
393 13707, 2024.
- 394 Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. Stanza:
395 A Python natural language processing toolkit for many human languages. In *Proceed-*
396 *ings of the 58th Annual Meeting of the Association for Computational Linguistics: System*
397 *Demonstrations*, pp. 101–108, 2020.

- 398 Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese
399 BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural*
400 *Language Processing and the 9th International Joint Conference on Natural Language Processing*
401 *(EMNLP-IJCNLP)*, pp. 3982–3992, 2019.
- 402 Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications,
403 proofs, and abductive statements over natural language. In *Findings of the Association for*
404 *Computational Linguistics: ACL-IJCNLP 2021*, pp. 3621–3634, 2021.
- 405 Xiao-Wen Yang, Jie-Jing Shao, Lan-Zhe Guo, Bo-Wen Zhang, Zhi Zhou, Lin-Han Jia, Wang-
406 Zhou Dai, and Yu-Feng Li. Neuro-symbolic artificial intelligence: Towards improving the
407 reasoning abilities of large language models. In *Proceedings of the Thirty-Fourth International*
408 *Joint Conference on Artificial Intelligence, IJCAI-25*, pp. 10770–10778, 2025. doi: 10.24963/
409 ijcai.2025/1195. Survey Track.
- 410 Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. SatLM: Satisfiability-aided language
411 models using declarative prompting. In *Advances in Neural Information Processing Systems*,
412 volume 36, pp. 45584–45607, 2023.
- 413 Wenpeng Yin, Dragomir Radev, and Caiming Xiong. DocNLI: A large-scale dataset for
414 document-level natural language inference. In *Findings of the Association for Computational*
415 *Linguistics: ACL-IJCNLP 2021*, pp. 4913–4922, 2021.

A Appendix

A.1 Logification Prompt

We use two logification prompts for converting natural language text into propositional logic. When not evaluating on the Alice dataset, we run the following few-shot, CoT prompt:

ROLE
You are a neuro-symbolic reasoning assistant specializing in translating natural language to zeroth-order propositional logic, aided by preprocessed OpenIE relation triples.

INTENDED TASK
Given natural language text T and POSSIBLY its Stanford OpenIE relation triples (you may NOT be given triples), extract:

1. PRIMITIVE PROPOSITIONS (P₁, P₂, ..., P_n)
 - Primitive, truth-evaluable statements that are not themselves connectives of finer propositions (e.g. no negated statements).
 - Non-redundant: avoid paraphrase duplicates; do not split into overlapping variants unless required by uncertainty rules
 - Include natural language meaning and textual evidence
 - Propositions must be specific, complete, and exhaustive for all constraints
 - CRITICAL: The proposition TEXT must be a simple declarative clause with no conditional words ("if", "when", "unless", "whenever", "not") or logical connectives ("and", "or").
2. CONSTRAINTS WITH WEIGHTS (C₁, C₂, ..., C_m)
 - Zero-order propositional formulas over {P₁, ..., P_n} suggested by the text
 - Each constraint has a llm_weight between 0 and 1 describing how strongly the text implies the formula holds
 - llm_weight = 1.0 means the constraint must hold
 - llm_weight = 0.5 means the constraint may be satisfied with 50% uncertainty
 - Include meaning, evidence, and reasoning from text
 - If contradictions exist, prioritize text intent; only include inconsistencies if genuinely intended
 - IMPORTANT: You must look at the primitive propositions and examine their translations. There may be common-sense semantic relationships between them. You must include constraints corresponding to these relationships with appropriate weights, including implied logical constraints.

METHODOLOGY
Let's work step-by-step as follows:

STEP 0: Review Exemplars

- Study examples for task intent and baseline quality expectations
- Use for guidance on ambiguous or interpretable text

STEP 1: Analyze Relation Triples

- If present, review triples to understand extracted entities and relationships
- CRITICAL: Triples are incomplete preprocessing hints. Follow this reconciliation protocol:
 1. Extract propositions from the original text first
 2. Use triples to recover missed arguments or entities
 3. Reject triples that introduce entities, relations, or facts absent from or contradictory to the text
 4. When triples conflict with text, always prioritize the text

STEP 2: Construct Primitive Propositions

```

477 - Extract atomic, truth-evaluable statements from text that cannot
478 further be broken down
479 - Ensure propositions are non-redundant and that they exhaust all
480 constraints
481 - Break down to genuinely atomic statements (no hidden logical
482 connectives)
483 - CRITICAL RULE: If text contains conditional or connective words
484 ("if", "when", "unless", "and", "or", "not"), extract component
485 clauses separately
486 - Example: "When Alice is focused, she completes her assignments on
487 time"
488   -> P_13: "Alice is focused" (simple declarative)
489   -> P_14: "Alice completes her assignments on time"
490       (simple declarative)
491   -> Then create constraint: C: P_13 ==> P_14 with weight 1.0
492 - Remove ALL modal/uncertainty words and use them to help inform the
493 weights of constraints: these are words indicating modality,
494 frequency, or strength such as always, never, usually, typically,
495 generally, often, frequently, sometimes, occasionally, rarely,
496 seldom, may, might, could, can, should, ought, encouraged,
497 discouraged, recommended, optional.
498
499 GUIDELINES:
500 - Use zeroth-order logic only (no quantifiers)
501 - Use consistent terminology: "primitive propositions" (matching JSON
502 field `primitive_props`)
503 - Flatten first-order-like statements: "Every student studies Tuesday"
504   -> identify all students (Alice, Bob) -> "Alice studies Tuesday /\ Bob
505       studies Tuesday"
506 - Ensure faithfulness to original text
507 - Extract implicit assumptions using text intent and common sense
508 - When triples conflict with text, prioritize text
509 - Constraints inform the primitive propositions, so update primitive
510   propositions when necessary
511 - Make sure to compare with the examples
512
513 GRAMMAR
514 Use standard zeroth-order propositional logic:
515 - Variables: P_1, P_2, ...
516 - Operators: not, and, or, implies, iff
517 - Use parentheses when needed to avoid ambiguity
518
519 OUTPUT
520 Output ONLY valid JSON following this schema:
521
522 {
523   "primitive_props": [
524     {
525       "id": "P_1",
526       "translation": <proposition meaning in natural language>,
527       "evidence": <location in text where proposition first appears;
528                  sentence indices are 0-based (first sentence is
529                  Sentence 0)>,
530       "explanation": <brief explanation for why this is atomic and
531                    cannot be broken down>
532     }, ...
533   ],
534   "constraints": [
535     {
536       "id": "C_1",
537       "formula": <propositional formula over P_i>,
538       "translation": <natural language translation; prefer brief direct
539                     quote or paraphrase>,
540       "llm_weight": <float 0.0-1.0>,
541       "evidence": <location in text; sentence indices are 0-based>,

```

```

542     "reasoning": <why this constraint has this weight>
543   }, ...
544 ]
545 }
546
547 If saving to disk, name the file "logified.json".
548
549 INPUT FORMAT
550 You will receive the original text T and OpenIE relation triples.
551 Sentence indices in triples are 0-based.
552
553 ORIGINAL TEXT:
554 <<<
555 [Original natural language text]
556 >>>
557
558 RELATION TRIPLES:
559 <<<
560 [JSON array where each triple is:
561   [subject, predicate, object, sentence_index]]
562 Example:
563 [[ "Alice", "likes", "Bob", 0], [ "Bob", "studies", "hard", 1]]
564 Sentence indices are 0-based (first sentence is index 0).
565 >>>

```

566 The prompt is followed by two examples (omitted for brevity). For the Alice dataset we run
567 the following one-shot prompt to retain the modal wording.

```

568 ROLE
569 You are a neuro-symbolic assistant translating natural language to
570 propositional logic.
571
572 TASK
573 Given text T and optional OpenIE triples which attempt to support T,
574 extract:
575 1. PRIMITIVE PROPOSITIONS (P_1, P_2, ...) - atomic, truth-evaluable
576    statements
577 2. CONSTRAINTS (C_1, C_2, ...) - propositional formulas with
578    llm_weight (0-1)
579
580 RULES
581 - Propositions must be simple declaratives that are atomic; they must
582   not have connectives in them (e.g. they cannot be conditionals or be
583   negations).
584 - Extract conditionals as separate propositions + implication
585   constraint
586 - Modal words (may, should, typically, sometimes, rarely): create TWO
587   propositions:
588   * P_modal: original with modal word
589   * P_base: without modal word
590   Then add constraints: P_modal@1.0, P_base@weight,
591   P_base==>P_modal@1.0
592
593 ID FORMAT (IMPORTANT)
594 - Proposition IDs must be strictly P_N where N is a number
595   (e.g., P_1, P_38)
596 - Do NOT use letter suffixes like P_38a, P_38b - use P_38, P_39, P_40
597   instead
598 - Constraint IDs must be strictly C_N where N is a number
599
600 WEIGHTS
601 - 1.0: must, cannot, never, always, original modal statements
602 - 1.0: deontic "may"/"permitted"/"allowed" (permission in
603   contracts/legal)
604 - 1.0: factual statements, obligations, and permissions that are

```

```

605   clearly stated in the text
606   - 0.85: should, required
607   - 0.65: usually, typically, generally
608   - 0.40: sometimes, might, encouraged, epistemic "may" (possibility)
609   - 0.25: rarely, seldom
610
611   GRAMMAR
612   ¬ (not), (and), (or), (implies), (iff)
613
614   OUTPUT (JSON only)
615   {
616     "primitive_props": [
617       {
618         "id": "P_1",
619         "translation": "...",
620         "evidence": "Sentence N",
621         "explanation": "..."
622       }
623     ],
624     "constraints": [
625       {
626         "id": "C_1",
627         "formula": "P_1",
628         "translation": "...",
629         "llm_weight": 1.0,
630         "evidence": "...",
631         "reasoning": "..."
632       }
633     ]
634   }

```

Note, this exemplar has also been omitted for brevity.

A.2 Logification JSON format

The final knowledge base as a JSON file has three components:

```

638 {"primitive_props": [
639   {"id": "P_1", "translation": "...",
640    "evidence": "...", "explanation": "..."}],
641  "hard_constraints": [
642    {"id": "C_1", "formula": "P_1 P_2",
643     "translation": "...", "evidence": "...",
644     "reasoning": "...", "llm-weight": 1.0,
645     "nli_entailment": 1.0,
646     "combined=(llm)*(nli)": 1.0}],
647  "soft_constraints": [
648    {"id": "C_5", "formula": "P_6",
649     "translation": "...", "evidence": "...",
650     "reasoning": "...", "llm-weight": 0.65,
651     "nli_entailment": 1.0,
652     "combined=(llm)*(nli)": 0.65, "weight": 0.65}]]}

```

The id field refers to the index in the enumeration of the proposition or constraint; the translation field refers to its meaning in natural language; the evidence field refers to the first source location in the text where it appears; and the explanation field provides a brief reasoning for the LLM's choice. Note, while we retain the soft constraints in the final logified file, for the purposes of SAT solving they are ignored. We keep them for the possibility of extending the solvers to accommodate soft probabilistic logic.

A.3 Hypothesis Generation Prompt

For the DocNLI dataset, we regenerate queries using an LLM to obtain 4-way classification labels (entailment, contradiction, uncertain, not_mentioned) that better exercise logical rather than textual entailment. The prompt is below.

You are generating test data for a logical inference system that uses formal propositional logic.

The system works by:

1. Extracting atomic propositions from the premise (simple statements without logical connectives)
2. Matching hypothesis text to these propositions using semantic similarity
3. Using a logic solver to check entailment
4. You generate statements built out of these propositions with varying degrees of complexity --- usually simple but o

Given the following premise, generate exactly {HYPOTHESES_PER_LABEL * 4} single-sentence hypotheses:

- {HYPOTHESES_PER_LABEL} ENTAILMENT hypotheses: Statements that are definitely TRUE based on the premise

- {HYPOTHESES_PER_LABEL} CONTRADICTION hypotheses: Statements that are definitely FALSE based on the premise

- {HYPOTHESES_PER_LABEL} UNCERTAIN hypotheses: Statements that COULD be true or false --- the premise is consistent wi

- {HYPOTHESES_PER_LABEL} NOT_MENTIONED hypotheses: See special rules below

CRITICAL RULES FOR ENTAILMENT HYPOTHESES:

1. USE EXPLICIT FACTS ONLY

Good: "Huey said she knew Francis since 1980"

Good: "Francis told Huey that Johnson feared the drug test"

Bad: "Huey recalls Francis implying..." (inference, not stated)

2. REFERENCE WHAT WAS SAID, DONE, OR REPORTED

Good: "X said that Y", "X testified that Y", "X did Y"

Bad: "X implied that Y", "X believed that Y" (unless explicitly stated)

3. COMBINE FACTS NATURALLY

Good: "X and Y" --- both facts stated

Good: "X, who did Y" --- relative clause combining facts

Good: "The [noun] that [fact1] also [fact2]"

4. USE NAMES, DATES, AND NUMBERS EXACTLY AS STATED

Good: Use "NBC-TV" if the text says "NBC-TV"

Bad: Paraphrase "NBC-TV" as "a television network"

CRITICAL RULES FOR CONTRADICTION HYPOTHESES:

1. USE LEXICALLY DISTINGUISHABLE ERRORS (the system uses word matching)

Good: Wrong proper nouns: "CBC-TV" instead of "NBC-TV"

Good: Wrong numbers/dates: "1985" instead of "1980"

Good: Wrong names: "Johnson" instead of "Lewis"

Good: Negated actions: "rejected" instead of "approved"

2. AVOID SUBTLE SEMANTIC SHIFTS

Bad: Changing only one adjective or adverb

Bad: Synonyms that are lexically similar

3. TYPES OF CONTRADICTION TO INCLUDE:

- Directly negate or reverse a stated fact

- Swap names, organizations, locations

- Change dates, numbers, quantities

- Attribute statements to the wrong person

CRITICAL RULES FOR UNCERTAIN HYPOTHESES:

722
723 The premise NEITHER confirms NOR denies these. Both true and false are possible.
724

725 1. USE UNDERSPECIFIED ASPECTS
726 Good: "Huey met Francis at a restaurant" (they met, but location not specified)
727 Good: "Johnson was nervous during the interview" (interview happened, emotion not stated)
728

729 2. COMBINE KNOWN FACTS WITH UNKNOWN DETAILS
730 Good: "Francis warned Huey before the test" (both exist, warning not mentioned)
731 Good: "The committee met twice before approving" (approval happened, count unknown)
732

733 3. WHAT MAKES IT UNCERTAIN (NOT CONTRADICTION):
734 - The premise doesn't rule it out
735 - It's plausible given what's stated
736 - Adding this fact wouldn't create a logical contradiction
737

738 =====
739 CRITICAL RULES FOR NOT_MENTIONED HYPOTHESES:
740 =====
741

742 Generate TWO types of not_mentioned hypotheses:
743

744 TYPE A --- HARD (same broad domain, but different entities/events):
745 Generate {HYPOTHESES_PER_LABEL // 2} hypotheses that are in the SAME DOMAIN as the premise
746 but about DIFFERENT specific entities, events, or facts NOT mentioned in the premise.
747 Good (sports premise): "Roger Federer withdrew from the French Open due to a knee injury."
748 (same domain: sports/tennis, but different people and events)
749 Good (legal premise): "The California Supreme Court overturned a ruling on eminent domain."
750 (same domain: law, but different court, state, and issue)
751 The key: shares general topic vocabulary but NO overlap with specific premise entities.
752

753 TYPE B --- EASY (completely different domain):
754 Generate {HYPOTHESES_PER_LABEL - HYPOTHESES_PER_LABEL // 2} hypotheses about COMPLETELY DIFFERENT topics.
755 Good: "The European Central Bank raised interest rates in March."
756 Good: "Toyota announced a recall of 500,000 vehicles."
757 Ensure ZERO lexical overlap with key premise terms.
758

759 ALL not_mentioned hypotheses must be realistic standalone statements.
760

761 =====
762 FORMATTING REQUIREMENTS:
763 =====
764

765 - Each hypothesis must be ONE SINGLE SENTENCE (10-30 words)
766 - Cover DIFFERENT aspects of the premise
767 - Use simple, direct language
768

769 =====
770 PREMISE:
771 =====
772 {premise}
773

774 =====
775 OUTPUT FORMAT (respond with this exact JSON structure, {HYPOTHESES_PER_LABEL} items per list):
776 =====
777 {{
778 "entailment": [
779 "First entailment combining explicit facts naturally.",
780 "Second entailment using different facts from the premise.",
781 "Third entailment covering another aspect of the premise.",
782 "Fourth entailment with exact names/dates/numbers from text."
783],
784 "contradiction": [
785 "First contradiction with wrong proper noun.",
786 "Second contradiction with wrong date or number.",

```

787         "Third contradiction that negates a stated fact.",
788         "Fourth contradiction with wrong attribution."
789     ],
790     "uncertain": [
791         "First uncertain: known event with unspecified detail.",
792         "Second uncertain: plausible extension of stated facts.",
793         "Third uncertain: combines known entities in unconfirmed way.",
794         "Fourth uncertain: reasonable detail not explicitly stated."
795     ],
796     "not_mentioned": [
797         "TYPE A (hard): same-domain statement about different entities.",
798         "TYPE A (hard): same-domain statement about different events.",
799         "TYPE B (easy): entirely different domain and topic.",
800         "TYPE B (easy): no lexical overlap with premise."
801     ]
802 }}

```

803 A.4 Query Translation Prompt

804 Given the top-10 retrieved propositions and a natural-language statement, we use the
805 prompt below on GPT-5 Nano to output a propositional formula or NONE. Note, the prompt
806 includes one exemplar but we have omitted it for brevity.

```

807 ROLE
808 You are a neuro-symbolic assistant translating natural language to
809 propositional logic.
810
811 TASK
812 Given text T and optional OpenIE triples which attempt to support T,
813 extract:
814 1. PRIMITIVE PROPOSITIONS (P_1, P_2, ...) - atomic, truth-evaluable
815    statements
816 2. CONSTRAINTS (C_1, C_2, ...) - propositional formulas with
817    llm_weight (0-1)
818
819 RULES
820 - Propositions must be simple declaratives that are atomic; they must
821   not have connectives in them (e.g. they cannot be conditionals or be
822   negations).
823 - Extract conditionals as separate propositions + implication
824   constraint
825 - Modal words (may, should, typically, sometimes, rarely): create TWO
826   propositions:
827   * P_modal: original with modal word
828   * P_base: without modal word
829   Then add constraints: P_modal@1.0, P_base@weight,
830   P_base==>P_modal@1.0
831
832 ID FORMAT (IMPORTANT)
833 - Proposition IDs must be strictly P_N where N is a number
834   (e.g., P_1, P_38)
835 - Do NOT use letter suffixes like P_38a, P_38b - use P_38, P_39, P_40
836   instead
837 - Constraint IDs must be strictly C_N where N is a number
838
839 WEIGHTS
840 - 1.0: must, cannot, never, always, original modal statements
841 - 1.0: deontic "may"/"permitted"/"allowed" (permission in
842   contracts/legal)
843 - 1.0: factual statements, obligations, and permissions that are
844   clearly stated in the text
845 - 0.85: should, required
846 - 0.65: usually, typically, generally
847 - 0.40: sometimes, might, encouraged, epistemic "may" (possibility)

```

848 - 0.25: rarely, seldom

849 A.5 Interesting case in LogicBench

850 An interesting use case for one reported failure from our pipeline arises in the following
 851 LogicBench example: “If Mason decides to leave his job, he will not receive any salary.” Our
 852 pipeline is particularly strict about logical form and treats *deciding to leave* and *having left* as
 853 distinct states, whereas the benchmark annotation appears to interpret the statement more
 854 pragmatically. Under this stricter notion of logical entailment, it is possible that Mason has
 855 decided to leave but has not yet acted on that decision, so his employment status is not
 856 definitively determined.

857 A.6 Results on other datasets

858 A.6.1 LogicBench

859 In this data set each query is either entailed or not entailed. In Table 3, for entailment
 860 examples (Yes), *Shortcuts*, *Expand*, and *Enrich* each improve accuracy by about +0.04, while
 861 *OpenIE* has a small negative effect. For not-entailment examples (No), *Enrich* has the
 862 largest negative effect, followed by *Expand* and *Shortcuts*, while *OpenIE* is near zero. The
 863 experimental design is valid (orthogonality checks pass) Effect directions and magnitudes
 864 with CIs “No significant main effects were detected, though T1 and T2 showed suggestive
 865 opposite effects on non-entailment questions (± 7.5

866 We conducted a 2^{4-1}_{IV} fractional factorial experiment (n=320 questions) to assess four pipeline
 867 components: shortcut detection (Q_1), query expansion (Q_2), OpenIE extraction (T_1), and
 868 knowledge enrichment (T_2). No significant main effects were detected at $\alpha = 0.05$, though T_2
 869 (*Enrich*) showed a suggestive negative effect on non-entailment questions (-15.5 % accuracy),
 870 while T_1 (*OpenIE*) had a negligible effect (+0.1%).

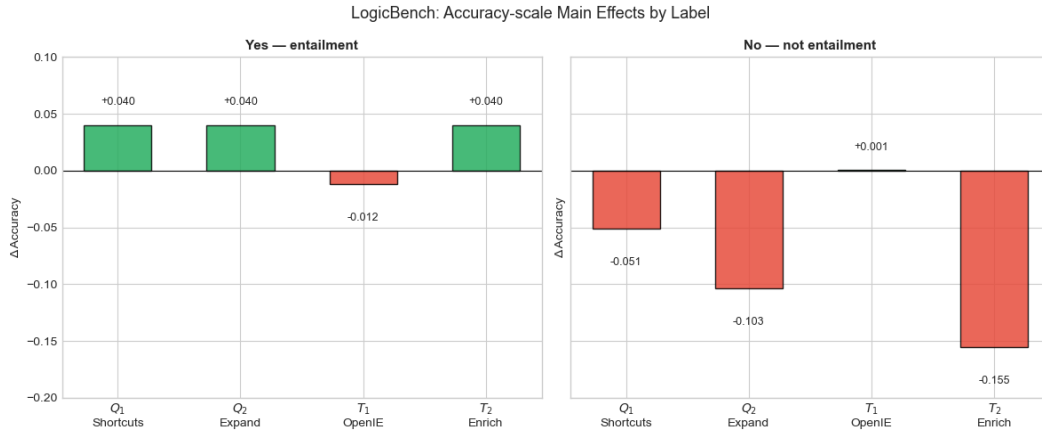


Figure 3: Main effects on accuracy for each factor in LogicBench, separated by label. Positive values indicate that switching a factor from its low level to its high level increases accuracy, while negative values indicate the opposite.